



School of Artificial Intelligence

(FARIDABAD – DELHI NCR)

B. Tech in Artificial intelligence and Data Science

23BIO112 – Introduction to Biological Data

23AID101 – Computational Thinking

END SEMESTER PROJECT EVALUATION REPORT

DL.AI.U4AID24114 – Kalyani Agarawal

DL.AI.U4AID24116 – Kavaru Amith Vignesh

DL.AI.U4AID24120 – Madesh KK

DL.AI.U4AID24125 - Parima Verma

2024-2025 (ODD SEMESTER-1)

Comprehensive Analysis and Visualization of PCOS Dataset: A Comparative Study of Normal and Cystic Ovaries Ultrasound Images Using Python

Date: 06-Jan-2025

1.Abstract:

This report presents a comprehensive analysis of the PCOS (Polycystic Ovary Syndrome) dataset, incorporating ultrasound image analysis alongside numerical and categorical data exploration to identify patterns and correlations that enhance the understanding of PCOS.

The image analysis component focuses on ultrasound images of ovaries to differentiate between PCOS and non-PCOS cases. Various preprocessing techniques, including cropping, zooming, augmentation, edge detection, and contour drawing, were applied to extract and visualize the ovarian regions. Pixel intensity distributions were compared between normal and PCOS ovaries, with the use of heatmaps, follicle overlay, and intensity analysis providing visual insights into the structural differences between cystic and healthy ovaries.

Following the image analysis, the report explores the numerical and categorical data, which includes attributes such as age, weight, height, BMI, hormonal levels, menstrual cycle characteristics, and lifestyle factors, recorded for both PCOS and non-PCOS patients. The primary objective is to investigate how factors like age, BMI, hormonal imbalances (FSH, LH), follicle counts, and metabolic health indicators (blood sugar, blood pressure) correlate with the occurrence of PCOS. Python libraries such as NumPy and Pandas were used for data manipulation, while Matplotlib and Seaborn facilitated visualization through histograms, scatter plots, and box plots. The findings reveal significant differences in BMI, exercise habits, hormonal imbalances, and metabolic health between PCOS and non-PCOS patients. Lifestyle factors like fast food consumption and regular exercise were also identified as influencing the prevalence of PCOS.

By integrating both image analysis and numerical data exploration, this report provides a comprehensive approach to understanding PCOS, offering valuable insights for future research and clinical applications.

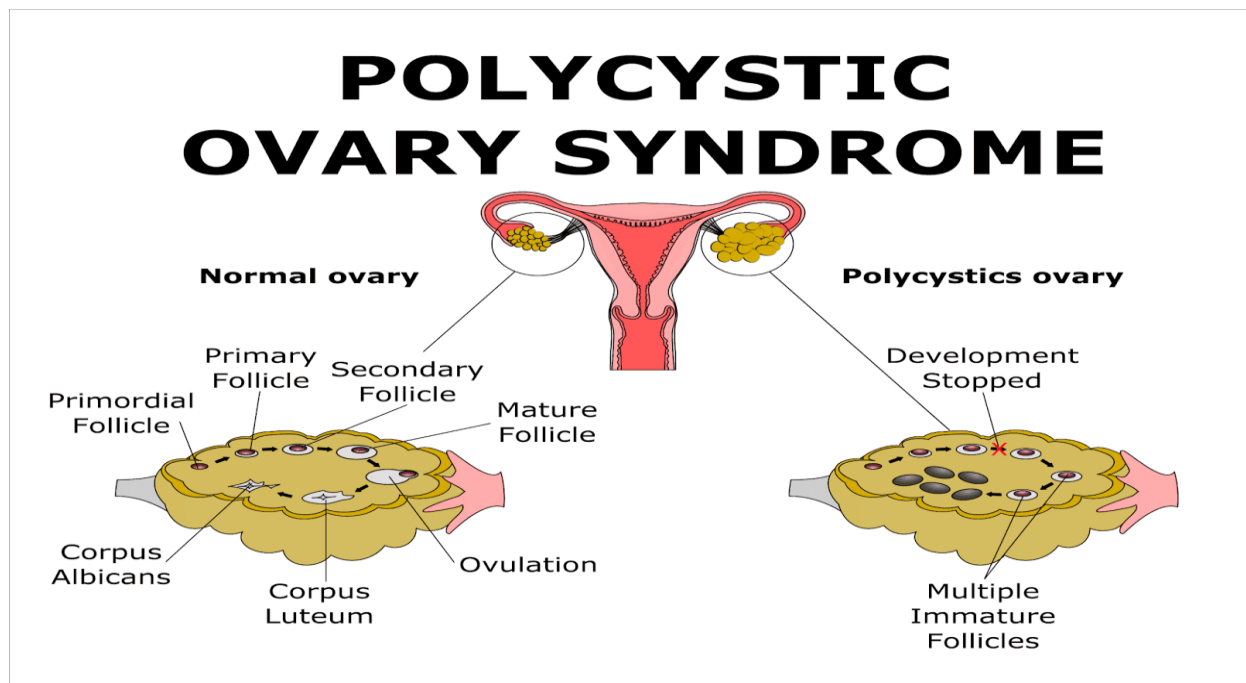
2. Table of Contents:

1. Introduction	(4)
1.1 Overview of Biological Data	
1.2 Role of Python Programming in Analysing Biological Data	
2. Objectives of the Project	(5)
3. Dataset Description	(6)
3.1 Source of Data	
3.2 Type of Data	
3.3 Understanding the Dataset	
3.4 Size of the Dataset	
4. Methodology	(9)
4.1 Tools and Libraries Used	
4.2 Steps Followed	
5. Implementation	(13)
6. Challenges Faced During Implementation	
of Ultrasound Images and Dataset Analysis	(28)
7. Results Summary	(29)
8. Discussion	(30)
8.1 Interpretation of Results	
8.2 Significance in Biological Context	
8.3 Limitations	
9. Conclusion	(31)
9.1 Summary of Work	
9.2 Key Takeaways	
9.3 Potential Applications	
9.4 Suggestions for Future Work	
10. References	(33)
11. Appendix	(35)

3.Introduction:

Overview of Biological Data:

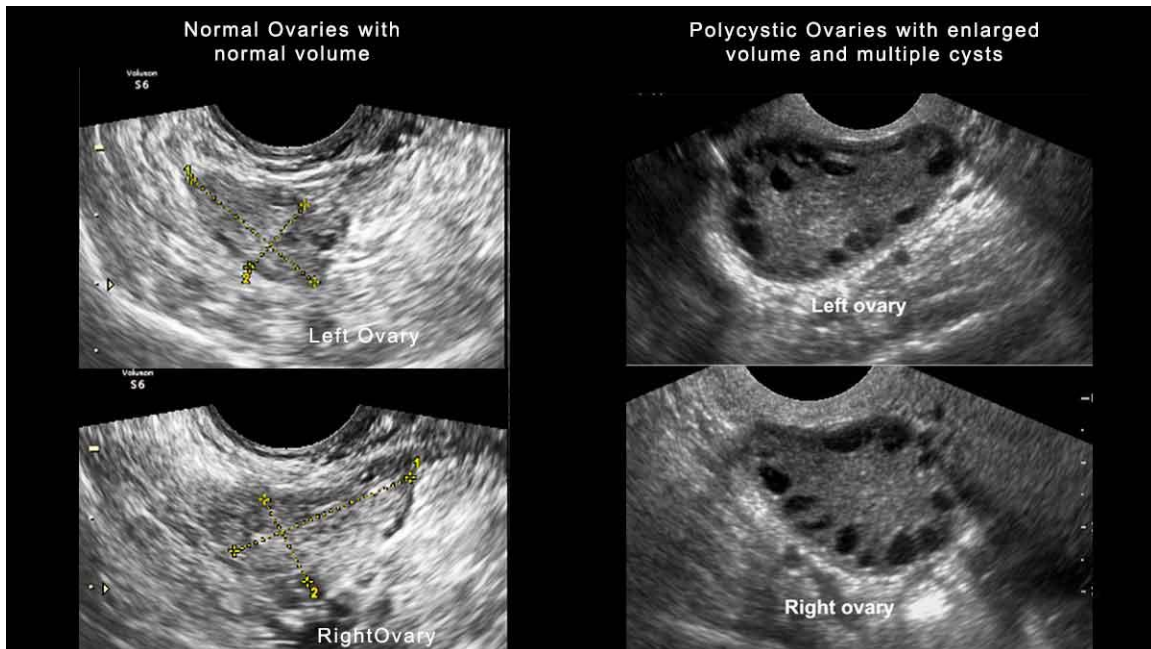
Biological data refers to information about the biological and physiological characteristics of living organisms, including variables such as age, weight, height, hormonal levels, and various health metrics. These variables are essential for understanding disease patterns, biological processes, and health outcomes. In medical research, biological data plays a critical role in analyzing disease trends, predicting health risks, and supporting clinical decision-making.



This project focuses on analyzing a PCOS (Polycystic Ovary Syndrome) dataset, which includes key attributes such as hormonal levels, BMI, menstrual cycle irregularities, and lifestyle factors like exercise and diet. Alongside this numerical data, ultrasound images of ovaries are also examined to identify visual patterns and characteristics associated with PCOS. The image analysis aims to distinguish between cystic ovaries, which are indicative of PCOS, and healthy ovaries. This approach adds an additional layer of insight, complementing the clinical data and providing a more comprehensive understanding of PCOS.

Role of Python Programming in Analysing Biological Data:

Python programming has become an essential tool in the analysis of biological data due to its flexibility, simplicity, and powerful libraries for data manipulation, analysis, and visualization. Libraries like NumPy and Pandas are utilized for efficient data cleaning and manipulation, while Matplotlib and Seaborn help in visualizing data trends. Python's OS and OpenCV libraries streamline bioinformatics tasks. OS manages file operations, while OpenCV handles image processing, such as resizing and segmentation, making them essential for organizing and analyzing biological data like ultrasound images.



4.Objectives of the Project:

1. Comprehensive Ultrasound Image Analysis

Analyse and compare ultrasound images of normal and PCOS ovaries using image processing techniques such as cropping, augmentation, edge detection, and pixel intensity comparison. The goal is to differentiate between normal and cystic ovaries by highlighting key structural differences using visual tools like heatmaps and follicle overlays.

2. Data Exploration and Correlation with PCOS Factors

Clean and preprocess the PCOS dataset, which includes information on age, BMI, weight, hormonal levels, and menstrual cycles. Using Python libraries like NumPy and Pandas, the objective is to explore correlations between these factors and their relationship with the occurrence of PCOS, providing insights into the underlying causes and risk factors of the condition.

3. Visualizing and Understanding Structural Differences in Ovarian Health

Utilize image transformation techniques such as binary, grayscale, and RGB formatting to visually represent and compare structural differences between normal and PCOS ovaries. The objective is to enhance the understanding of follicle distribution and intensity variations, thereby contributing to more accurate and effective diagnostic methods for PCOS detection and treatment planning.

5.Dataset Description

A	B	C	D	E	F	G	H	I	J	K	L
Sl. No	Patient File No.	PCOS (Y/N)	Age (yrs)	Weight (Kg)	Height(Cm)	BMI	Blood Group	Pulse rate(bpm)	RR (breaths/min)	Hb(g/dl)	Cycle(R/l)
1	10001	0	28	44.6	152	19.3	15	78	22	10.48	2
2	10002	0	36	65	161.5	24.9212		74	20	11.7	2
3	10003	1	33	68.8	165	25.2709	11	72	18	11.8	2
4	10004	0		65	148	29.6749	13	72	20	12	2
5	10005	0	25	52	161	20.061	11	72	18		2
6	10006	0	36	74.1	165	27.2176	15	78	28	11.2	2
7	10007	0	34	64	156	26.2985	11	72	18	10.9	2
8	10008	0	33	58.5	159	23.1399	13	72	20	11	2
9	10009	0	32	40	158	16.0231	11	72	18	11.8	2
10	10010	0		52	150	23.1111	15	80	20	10	4
11	10011	0	20	71	163		15	80	20	10	2
12	10012	0	26	49	160	19.1406	13	72	20	9.5	2
13	10013	1	25	74	152	32.0291	17	72	18	11.7	4
14	10014	0	38	50	152	21.6413	13	74	20	12.1	2
15	10015	0	34	57.3	162	21.8336	13	74	22	11.7	2
16	10016	0	38	80.5	154	33.9433	13	78	22	11.4	2
17	10017	0	29	43	148		13	80		11.1	2
18	10018	0	36	69.2	160	27.0313	13	72	18	10.8	2
19	10019	0	31	52.4	159	20.727	17	72	18		2
20	10020	1	30	85	165	31.2213	16	72	18	12.5	4
21	10021	0	25		156	26.2985	11	70	18	11.2	2
22	10022	0	38	50	156	20.5457	15	72	18	11	4
23	10023	0	34	64.2	155	26.7222	15	74	20	12.1	2
24	10024	0	28	65	152	28.1337	13	74	22	10.5	2

1. Source of Data: The dataset is sourced from **Kaggle**, a platform providing diverse datasets for research and analysis.

i.Numerical/Categorical data:

<https://www.kaggle.com/code/mahikkaaa/pcos-prediction-and-analysis/notebook>

ii. Imaging Data:

<https://www.kaggle.com/datasets/shnotweta/2000-images-of-ultrasound-for-pcos/code>

2. Type of Data:

- **File Format:** ZIP (Contains ultrasound images) & CSV (Comma-Separated Values)

- **Biological Nature:**

The project uses two separate datasets: one containing ultrasound images of normal and PCOS-affected ovaries to identify cystic changes, and another with physical, hormonal, and lifestyle data for diagnosing and analyzing PCOS.

3. Understanding the Dataset

- **Understanding the Dataset**

This project utilizes two separate datasets: one for clinical and lifestyle data related to Polycystic Ovary Syndrome (PCOS), and another for ultrasound images of ovaries, both normal and PCOS-affected. Together, they provide a multi-dimensional view of the condition, combining statistical analysis with image processing techniques to enhance understanding, prediction, and diagnosis of PCOS.

Clinical and Lifestyle Data:

- **Key Attributes:**
 - **Demographic Information:** Includes attributes such as age, weight, height, and BMI (Body Mass Index), which are essential in understanding physical characteristics and overall health.
 - **Hormonal Levels:** Important hormonal markers such as FSH (Follicle Stimulating Hormone) and LH (Luteinizing Hormone) are included. These are often imbalanced in PCOS patients, contributing to irregular cycles and ovulation issues.
 - **Menstrual Cycle Characteristics:** Information regarding cycle length, frequency, and irregularities is captured to assess its relationship with PCOS.

- **Lifestyle Factors:** The dataset also records lifestyle-related factors such as regular exercise, fast food consumption, and weight gain, which are known to influence the onset and severity of PCOS.
- **Metabolic Health Indicators:** Variables like blood pressure, blood sugar (RBS), and other health indicators are included, helping to understand the metabolic effects of PCOS.

Ultrasound Image Data

- **Key Features:**
 - **Image Types:** The dataset includes images of ovaries showing cystic structures (common in PCOS) and healthy ovaries, which are important in detecting key visual markers associated with the condition.
 - **PCOS-Related Indicators:** Ultrasound images reveal structural differences in ovaries that indicate PCOS, such as cystic formations, multiple follicles, and an increase in ovarian volume, which are hallmarks of the syndrome.
 - **Image Resolution:** The images are of high quality, with sufficient detail to allow for accurate image preprocessing and analysis. This makes them suitable for machine learning applications such as classification and pattern recognition.

Combined Dataset for PCOS Analysis

- **Integrating Both Data Types:**
 - By combining clinical data and ultrasound image data, this project provides a more holistic view of PCOS. Clinical data offers valuable insights into the biological and lifestyle factors that contribute to PCOS, while ultrasound image data enhances the understanding of the condition's structural aspects.
 - This dual approach allows for both quantitative analysis (e.g., correlations between BMI and hormonal levels) and qualitative analysis (e.g., identification of cysts and ovarian abnormalities in images), improving the accuracy and depth of the PCOS diagnosis process.

In conclusion, these datasets provide a rich source of information for understanding, predicting, and diagnosing PCOS. By integrating both clinical and image-based data, this project aims to

deliver a more comprehensive approach to PCOS analysis and support the development of more effective diagnostic tools.

4. Size of the Dataset:

Ultrasound Image Data

Number of Images: 2000(for simplicity we reduced it to 1000)

The ultrasound image dataset consists of 2000 images, each depicting either a normal ovary or one affected by PCOS. These images are crucial for visual analysis to identify structural markers and differences between healthy and cystic ovaries, contributing to a more comprehensive understanding of PCOS through image-based diagnostics.

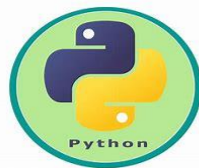
Numerical/Categorical Data

Rows: 541

Columns: 43

The numerical and categorical dataset includes 541 entries, each representing a different individual, with 43 attributes such as age, weight, hormonal levels, lifestyle factors, and menstrual cycle characteristics. These features are essential for understanding the relationships between personal and lifestyle factors and the occurrence of PCOS.

6.Methodology



1. Tools and Libraries Used

The following libraries and tools were employed throughout the project to manage data processing, analysis, and visualization:

- **NumPy:** Utilized for numerical computations, array manipulations, and statistical operations.
- **Pandas:** Used for efficient data manipulation, including loading datasets, cleaning data, handling missing values, and managing DataFrame structures.
- **Matplotlib:** Essential for data visualization, enabling the creation of histograms, scatter plots, and box plots.
- **Seaborn:** Leveraged for advanced visualizations such as heatmaps and correlation matrices.
- **OpenCV (cv2):** Applied for image processing tasks including edge detection, cropping, and zooming in on ultrasound images.

2. Steps Followed

Step 1: Data Preprocessing

- **Loading Datasets:** The CSV data containing numerical and categorical features was loaded using Pandas, while ultrasound images were imported using zipfile and Google Colab's import functions.
- **Data Cleaning:** Missing values were handled by imputing numerical data with mean values and categorical data with mode values using Pandas.
- **Outlier Handling:** Outliers were identified and removed using the Interquartile Range (IQR) method to ensure data integrity.

Step 2: Data Exploration and Statistical Analysis

- **Exploring Relationships:** NumPy was employed for statistical analyses such as mean, variance, and correlation between key variables like age, BMI, and hormonal levels, helping to understand PCOS-related factors.
- **Descriptive Statistics:** Pandas was used to generate statistical summaries, including mean and standard deviation for numerical data.

Step 3: Visualization

- **Data Visualization:** Matplotlib and Seaborn were used for visualizing the data:

Histograms and box plots were created to explore the distribution of continuous variables such as age, BMI, and hormone levels.

Scatter and pair plots were generated to examine relationships between pairs of variables like BMI vs. age.

Correlation heatmaps were plotted to analyze the strength of relationships between features in the dataset.

Step 4: Ultrasound Image Analysis

- **Image Preprocessing:** OpenCV was used to preprocess ultrasound images by isolating the ovarian region, applying transformations (such as zoom, rotate, and flip), and detecting edges using Canny edge detection.
- **Feature Extraction:** Structural differences between normal and PCOS ovaries were detected by identifying contours and highlighting follicles on ultrasound images.
- **Intensity Comparison:** Pixel intensity analysis was performed to compare the intensity of normal versus cystic ovaries, and histograms were generated for visual comparison.

By following these steps, the project aimed to integrate numerical and visual data analysis, providing a deeper understanding of PCOS and its correlations.

3. Workflow Breakdown

1. Read and Explore CSV File

Objective: Load and analyze the dataset, focusing on extracting counts of normal and PCOS cases.

- Steps: Use Pandas to load the dataset and count the number of "Yes" and "No" PCOS cases.

2. Image Preprocessing and Augmentation

Objective: Prepare ovarian images (normal vs. PCOS) for analysis.

- Steps: Isolate ovarian regions, apply augmentations like flipping and rotating, and perform edge detection and contour drawing.

3. Compare Pixel Intensity (Normal vs. PCOS)

Objective: Analyse and compare pixel intensity distributions between normal and PCOS ovaries.

- Steps: Convert images to grayscale, analyse histograms, and visualize intensity differences.

4. Image Transformation (Normal and PCOS)

Objective: Transform images into different formats and visualize the results.

- Steps: Convert images to binary, grayscale, and RGB formats, then visualize using Matplotlib.

5. Heatmap Comparison (Normal vs. PCOS)

Objective: Create heatmaps to compare follicle distribution in normal and PCOS ovaries.

- Steps: Calculate follicle intensities, plot heatmaps using Seaborn, and overlay the PCOS image on the normal image.

6. Follicle Overlay and Highlighting

Objective: Highlight follicle points and compare normal vs. PCOS ovaries.

- Steps: Extract follicle coordinates and overlay them on the images, using distinct colors for normal and PCOS follicles.

7. Convert Image to DataFrame

Objective: Convert image pixel data into a tabular format for analysis.

- Steps: Convert pixel data into arrays and create a DataFrame to compare pixel intensities of normal vs. PCOS ovaries.

8. Visualization and Analysis

Objective: Compare and visualize images in various formats.

- Steps: Create comparison plots, generate color maps to visualize ovarian structure differences, and overlay visualizations to highlight key differences.

4. Python Libraries Used

- **Data Handling:** pandas, numpy
- **Image Processing:** cv2 (OpenCV)
- **Visualization:** matplotlib, seaborn

7.Implementation

1. Mounting zip file in drive(1000 Images)

```
from google.colab import drive
drive.mount('/content/drive')
```

The code connects your Google Drive to the Colab environment, making your Drive files accessible in Colab. You'll be prompted to authenticate, and after that, your Drive files will be available under /content/drive.

2. Read and Explore CSV File(1000 Images)

```
import zipfile
zip_path = '/content/drive/MyDrive/archive2.zip'
extract_to = '/content/drive/MyDrive/extractfiles'
with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall(extract_to)
print("Files extracted to", extract_to)
```

Files extracted to /content/drive/MyDrive/extractfiles

This code extracts the contents of a ZIP file from a specified location in Google Drive. It first imports the zipfile module to handle ZIP file operations. The path to the ZIP file (archive2.zip) is defined in zip_path, and the destination folder for the extracted files is specified in extract_to. Using a context manager, the ZIP file is opened in read mode and all its contents are extracted to the extract_to folder. Finally, a confirmation message is printed to indicate that the files have been successfully extracted to the specified location.

```

#1. Total No. of Images
from google.colab import drive
import zipfile
import os
drive.mount('/content/drive')
zip_path = '/content/drive/MyDrive/archive2.zip'
extract_to = '/content/drive/MyDrive/extractfiles'
extracted_folder_pcos = '/content/drive/MyDrive/extractfiles/dataset/pcos'
extracted_folder_normal = '/content/drive/MyDrive/extractfiles/dataset/normal'
def count_images(folder_path):
    return sum(1 for f in os.listdir(folder_path) if f.lower().endswith((''.jpg', '.jpeg', '.png'))))
if os.path.exists(zip_path):
    with zipfile.ZipFile(zip_path, 'r') as zip_ref:
        zip_ref.extractall(extract_to)
    print("Files extracted to", extract_to)
    pcos_image_count = count_images(extracted_folder_pcos)
    normal_image_count = count_images(extracted_folder_normal)
    print("Total Images in PCOS folder:", pcos_image_count)
    print("Total Images in Normal folder:", normal_image_count)
    print("Total Images in the dataset:", pcos_image_count + normal_image_count)
else:
    print("Error: Zip file not found at", zip_path)

```

```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
Files extracted to /content/drive/MyDrive/extractfiles
Total Images in PCOS folder: 500
Total Images in Normal folder: 500
Total Images in the dataset: 1000

```

3. Image Preprocessing and Augmentation (1000 Images): This Python code is used to extract and analyze image files from a zip archive stored in Google Drive. It starts by importing necessary libraries like drive from Google Colab for accessing files on Google Drive, zipfile for extracting files from a zip archive, and os for interacting with the file system. The zip file is located at a specified path (zip_path), and the extraction destination is set to another folder (extract_to).

The function count_images(folder_path) is defined to count the number of image files (with extensions .jpg, .jpeg, or .png) in a given folder. The script first checks if the zip file exists. If it does, the zip file is opened, and its contents are extracted to the specified folder. After extraction, the number of image files in two subfolders—pcos and normal—is counted using the count_images function. The script then prints the total number of images in each folder, as well as the combined total from both folders. If the zip file does not exist at the specified location, an error message is printed.

```
#2. Image Preprocessing and Augmentation
import cv2
import os
import matplotlib.pyplot as plt

class OvarianImagePreprocessor:
    def __init__(self, image_path):
        self.image = cv2.imread(image_path)

    def crop_image(self, x, y, w, h):
        return self.image[y:y+h, x:x+w]

    def augment_image(self, cropped_image):
        return {'horizontal_flip': cv2.flip(cropped_image, 1),
                'vertical_flip': cv2.flip(cropped_image, 0),
                'rotation_90': cv2.rotate(cropped_image, cv2.ROTATE_90_CLOCKWISE),
                'scaling_150': cv2.resize(cropped_image, None, fx=1.5, fy=1.5)}

    def edge_detection(self, cropped_image):
        return cv2.Canny(cv2.cvtColor(cropped_image, cv2.COLOR_BGR2GRAY), 100, 200)

    def draw_contours(self, cropped_image, edges):
        contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
        return cv2.drawContours(cropped_image.copy(), contours, -1, (0, 255, 0), 2)
```

```
def visualize(self, cropped_image, augmented_images, edges, contour_image):
    fig, axes = plt.subplots(2, 3, figsize=(12, 8))
    images = [
        (cropped_image, "Cropped Image"), (augmented_images['horizontal_flip'], "Horizontal Flip"),
        (augmented_images['vertical_flip'], "Vertical Flip"), (augmented_images['rotation_90'], "Rotation 90°"),
        (augmented_images['scaling_150'], "Scaled (150%)"), (contour_image, "Contours")]
    for ax, (img, title) in zip(axes.ravel(), images):
        ax.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB) if len(img.shape) == 3 else img)
        ax.set_title(title)
        ax.axis('off')
    plt.tight_layout()
    plt.show()

def process_images_in_directory(directory_path):
    for filename in os.listdir(directory_path):
        if filename.endswith((".jpg", ".png")):
            image_path = os.path.join(directory_path, filename)
            preprocessor = OvarianImagePreprocessor(image_path)
            cropped_image = preprocessor.crop_image(100, 100, 300, 300)
            augmented_images = preprocessor.augment_image(cropped_image)
            edges = preprocessor.edge_detection(cropped_image)
            contour_image = preprocessor.draw_contours(cropped_image, edges)
            preprocessor.visualize(cropped_image, augmented_images, edges, contour_image)
```

```
def process_dataset(Dataset_Dir):
    for class_folder in ['pcos', 'normal']:
        class_path = os.path.join(Dataset_Dir, class_folder)
        if os.path.isdir(class_path):
            process_images_in_directory(class_path)

def main():
    dataset_directory = '/content/drive/MyDrive/extracted_files_2/dataset/'
    process_dataset(dataset_directory)

if __name__ == "__main__":
    main()
```

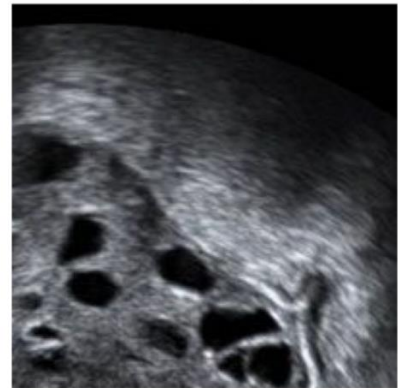
Cropped Image



Horizontal Flip



Vertical Flip



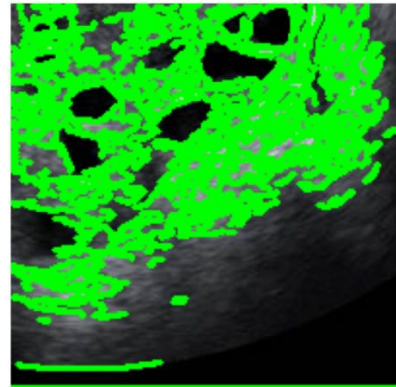
Rotation 90°



Scaled (150%)



Contours



4. Compare Pixel Intensity (Normal vs. PCOS) (1000 Images)

```
#3.Compare Pixel Intensity (Normal vs. PCOS)
import os
import numpy as np
import matplotlib.pyplot as plt
import cv2

class ImageProcessor:
    def __init__(self, normal_directory, pcos_directory):
        self.normal_directory = normal_directory
        self.pcos_directory = pcos_directory
        self.normal_images = [f for f in os.listdir(self.normal_directory) if f.endswith((''.jpg', '.png'))]
        self.pcos_images = [f for f in os.listdir(self.pcos_directory) if f.endswith((''.jpg', '.png'))]

    def process_image(self, image_path):
        img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
        if img is None:
            raise ValueError(f"Failed to load image: {image_path}")
        binary = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)[1]
        return binary.flatten()

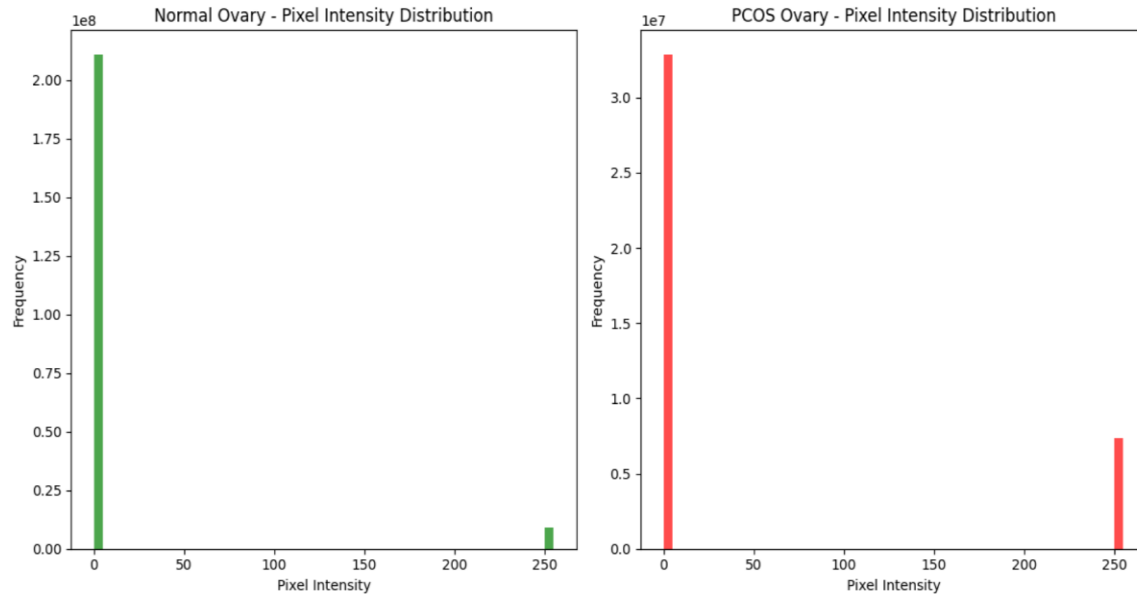
    def compare_pixel_intensity(self):
        normal_data_flat = np.concatenate([self.process_image(os.path.join(self.normal_directory, f)) for f in self.normal_images])
        pcpos_data_flat = np.concatenate([self.process_image(os.path.join(self.pcos_directory, f)) for f in self.pcos_images])
```

```
# Plot histograms
fig, axes = plt.subplots(1, 2, figsize=(12, 6))

axes[0].hist(normal_data_flat, bins=50, color='green', alpha=0.7)
axes[0].set_title("Normal Ovary - Pixel Intensity Distribution")
axes[0].set_xlabel("Pixel Intensity")
axes[0].set_ylabel("Frequency")

axes[1].hist(pcos_data_flat, bins=50, color='red', alpha=0.7)
axes[1].set_title("PCOS Ovary - Pixel Intensity Distribution")
axes[1].set_xlabel("Pixel Intensity")
axes[1].set_ylabel("Frequency")
plt.tight_layout()
plt.show()

normal_directory = '/content/drive/MyDrive/extractfiles/dataset/normal'
pcos_directory = '/content/drive/MyDrive/extractfiles/dataset/pcos'
processor = ImageProcessor(normal_directory, pcpos_directory)
processor.compare_pixel_intensity()
```



This Python code compares the pixel intensity distributions of images from two folders: one containing images labeled as "normal" and the other containing images labeled as "PCOS." The code defines a class `Image-Processor` that initializes with two directory paths—one for normal images and the other for PCOS images. Inside the class, the `process_image` method reads each image in grayscale and converts it into a binary format using a thresholding technique. The pixel intensities of each processed image are then flattened into a 1D array.

The `compare_pixel_intensity` method concatenates the flattened pixel intensities of all images in both the normal and PCOS directories. It then generates two histograms: one for the normal images and one for the PCOS images, to visualize the distribution of pixel intensities. The histograms are plotted side by side, with the normal images shown in green and the PCOS images in red. The pixel intensity distributions are presented with labels and titles, making it easy to compare how the two sets of images differ in terms of pixel intensities. The plot is displayed using `matplotlib`.

5. Image Transformation (Normal and PCOS) (1000 Images)

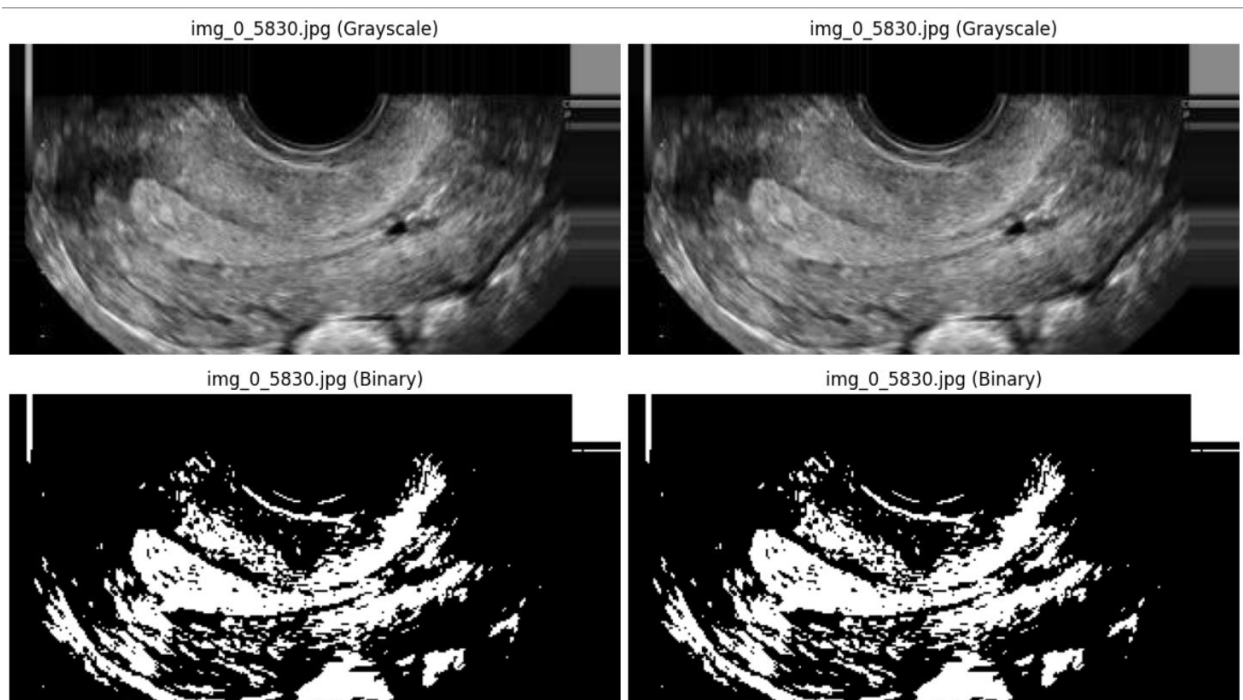
This Python code compares images of normal and PCOS ovaries by visualizing them in multiple stages: RGB, Grayscale, and Binary formats. The function `process_images` takes two directory paths as input—one for normal images and another for PCOS images. It first sorts the images in both

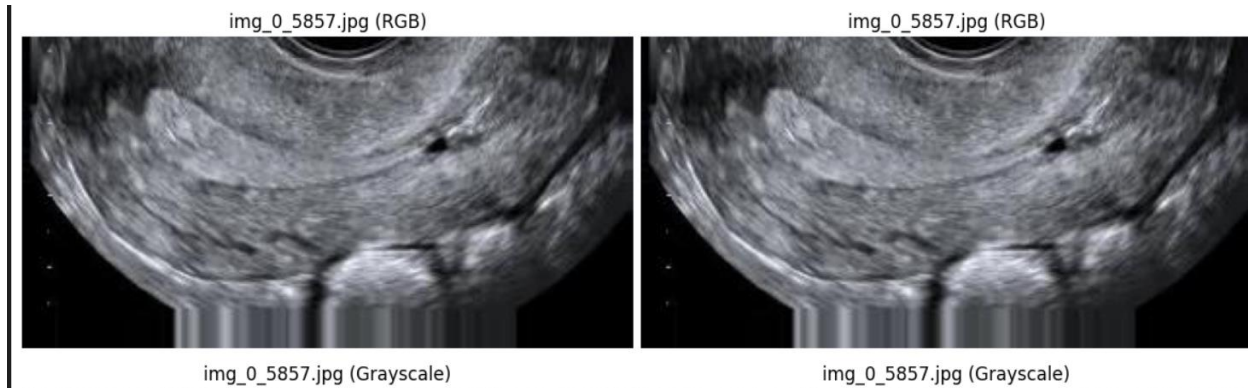
directories and then processes each corresponding pair of normal and PCOS images. For each image pair, the code reads the images using `cv2.imread`, converts them into grayscale, and then applies binary thresholding to create binary images.

The function then generates a 3x2 grid of subplots for each image pair, where:

1. The first column displays the original RGB images of the normal and PCOS ovaries.
2. The second column shows the grayscale versions of the images.
3. The third column shows the binary versions of the images after thresholding.

Each subplot is labeled with the image file name and the format (RGB, Grayscale, Binary). The images are displayed without axes for a clearer view. The main function initializes the directory paths and calls `process_images` to generate and display the comparisons. The layout of the plots is adjusted to ensure that the subplots are neatly spaced. Finally, the main function is called to execute the process.





#4. Image Comparison of Normal and PCOS Ovaries

```
import cv2
import matplotlib.pyplot as plt
import os

def process_images_in_directory(normal_directory, pcos_directory):
    normal_images = sorted(os.listdir(normal_directory))
    pcos_images = sorted(os.listdir(pcos_directory))

    for normal_image, pcos_image in zip(normal_images, pcos_images):
        normal_image_path = os.path.join(normal_directory, normal_image)
        pcos_image_path = os.path.join(pcos_directory, pcos_image)

        normal_image = cv2.imread(normal_image_path)
        pcos_image = cv2.imread(pcos_image_path)

        normal_gray = cv2.cvtColor(normal_image, cv2.COLOR_BGR2GRAY)
        pcos_gray = cv2.cvtColor(pcos_image, cv2.COLOR_BGR2GRAY)

        _, normal_binary = cv2.threshold(normal_gray, 127, 255, cv2.THRESH_BINARY)
        _, pcos_binary = cv2.threshold(pcos_gray, 127, 255, cv2.THRESH_BINARY)

        fig, axes = plt.subplots(3, 2, figsize=(12, 10))

        axes[0, 0].imshow(cv2.cvtColor(normal_image, cv2.COLOR_BGR2RGB))
        axes[0, 0].set_title(f'{normal_image}')
        axes[0, 0].axis('off')
```

```

axes[0, 0].imshow(cv2.cvtColor(normal_image, cv2.COLOR_BGR2RGB))
axes[0, 0].set_title(f'{normal_image}')
axes[0, 0].axis('off')

axes[0, 1].imshow(cv2.cvtColor(pcos_image, cv2.COLOR_BGR2RGB))
axes[0, 1].set_title(f'{pcos_image}')
axes[0, 1].axis('off')

axes[1, 0].imshow(normal_gray, cmap='gray')
axes[1, 0].axis('off')

axes[1, 1].imshow(pcos_gray, cmap='gray')
axes[1, 1].axis('off')

axes[2, 0].imshow(normal_binary, cmap='gray')
axes[2, 0].axis('off')

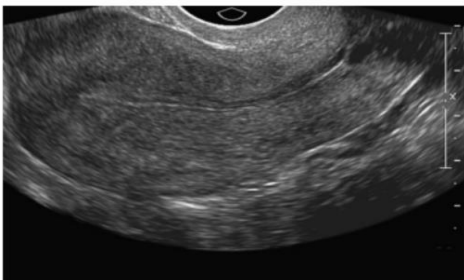
axes[2, 1].imshow(pcos_binary, cmap='gray')
axes[2, 1].axis('off')

plt.tight_layout()
plt.show()

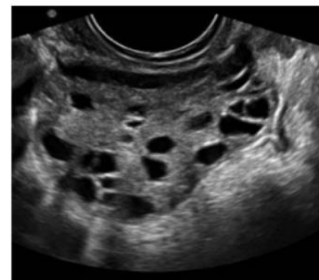
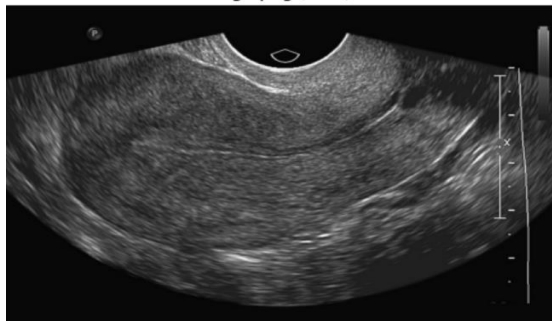
def main():
    normal_directory = '/content/drive/MyDrive/extractfiles/dataset/normal'
    pcos_directory = '/content/drive/MyDrive/extractfiles/dataset/pcos'
    process_images_in_directory(normal_directory, pcos_directory)

if __name__ == "__main__":
    main()

```



img2.png (RGB)



img10.jpg (RGB)



6. Heatmap Comparison (Normal vs. PCOS) (1000 Images)

This Python code performs a heatmap-based comparison between images of normal and PCOS ovaries, visualizing the differences in pixel intensities. The `PCOSComparison` class initializes with two directories—one containing normal ovary images and the other containing PCOS ovary images. It also sets a target image size for uniform comparison.

The method `process_images_in_directory` reads and processes images from both directories, ensuring the number of images in both directories match. For each image pair, it calls the `compare_images` method, which reads the images in grayscale, resizes them to the target size, and then calls `plot_heatmaps` to generate heatmaps.

In `plot_heatmaps`, three subplots are created for each image pair:

1. The first heatmap visualizes the normal ovary image with a blue color map.
2. The second heatmap visualizes the PCOS ovary image with a red color map.
3. The third heatmap shows the absolute difference between the normal and PCOS images using a cool-warm color map, highlighting the areas of difference.

Each subplot is labeled with the respective image's filename. Finally, the heatmaps are displayed using `matplotlib` and `seaborn`, providing a visual comparison of the normal and PCOS images and their differences. The `process_images_in_directory` method is called to start processing and displaying the comparisons.

```

#5.Heatmap Comparison (Normal vs. PCOS)
import cv2
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import os

class PCOSComparison:
    def __init__(self, normal_directory, pcos_directory):
        self.normal_directory = normal_directory
        self.pcos_directory = pcos_directory
        self.target_size = (256, 256)

    def process_images_in_directory(self):
        normal_images = sorted(os.listdir(self.normal_directory))
        pcos_images = sorted(os.listdir(self.pcos_directory))
        if len(normal_images) != len(pcos_images):
            print("Warning: Mismatch in number of images between normal and PCOS directories.")
        for normal_file, pcos_file in zip(normal_images, pcos_images):
            normal_path = os.path.join(self.normal_directory, normal_file)
            pcos_path = os.path.join(self.pcos_directory, pcos_file)
            print("Processing:", normal_file, "vs.", pcos_file)
            self.compare_images(normal_path, pcos_path)

    def compare_images(self, normal_path, pcos_path):
        normal_image = cv2.imread(normal_path, cv2.IMREAD_GRAYSCALE)
        pcos_image = cv2.imread(pcos_path, cv2.IMREAD_GRAYSCALE)

        if normal_image is None or pcos_image is None:
            print("Error loading images:", normal_path, "or", pcos_path)
            return

```



```

normal_image = cv2.resize(normal_image, self.target_size, interpolation=cv2.INTER_AREA)
pcos_image = cv2.resize(pcos_image, self.target_size, interpolation=cv2.INTER_AREA)
self.plot_heatmaps(normal_image, pcos_image, normal_path, pcos_path)

def plot_heatmaps(self, normal_image, pcos_image, normal_path, pcos_path):
    plt.figure(figsize=(15, 7))
    plt.subplot(1, 3, 1)
    sns.heatmap(normal_image, cmap="Blues", cbar=True)
    plt.title("Normal Ovary: " + os.path.basename(normal_path))

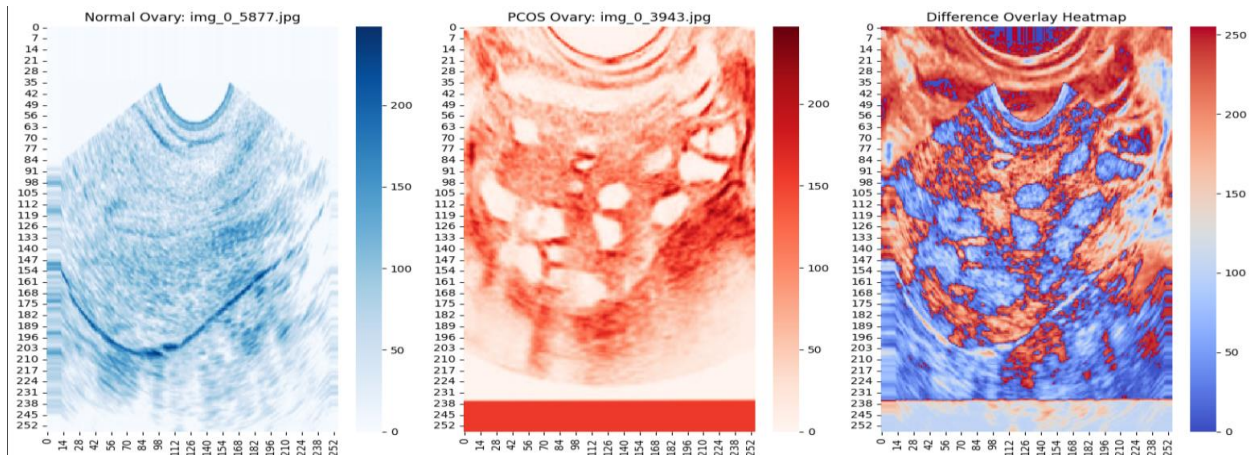
    plt.subplot(1, 3, 2)
    sns.heatmap(pcos_image, cmap="Reds", cbar=True)
    plt.title("PCOS Ovary: " + os.path.basename(pcos_path))

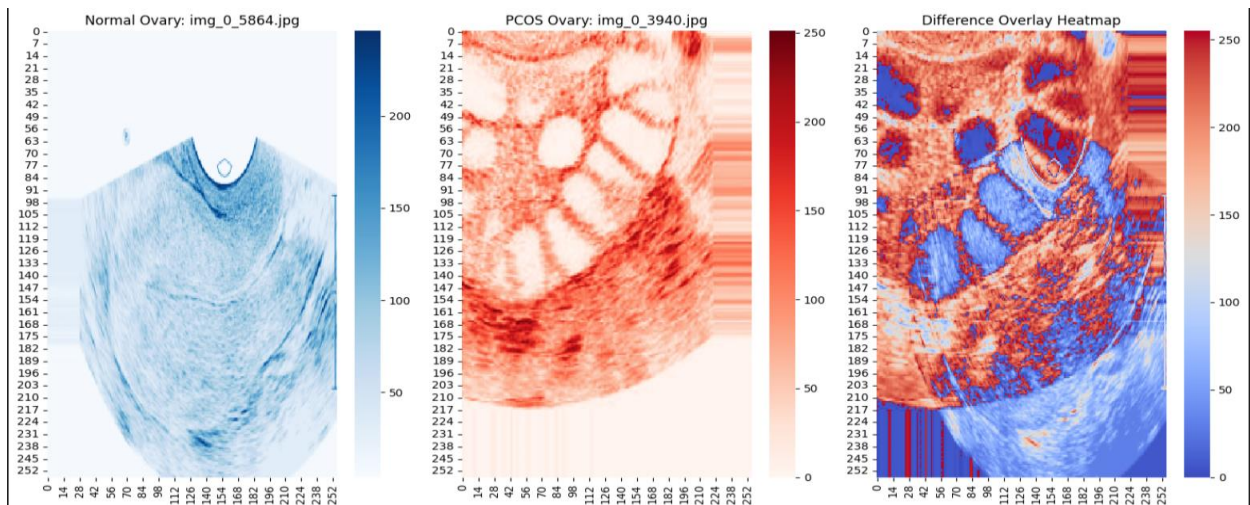
    difference = np.abs(normal_image - pcos_image)
    plt.subplot(1, 3, 3)
    sns.heatmap(difference, cmap="coolwarm", cbar=True)
    plt.title("Difference Overlay Heatmap")

    plt.tight_layout()
    plt.show()

normal_directory = '/content/drive/MyDrive/extractfiles/dataset/normal'
pcos_directory = '/content/drive/MyDrive/extractfiles/dataset/pcos'
comparison = PCOSComparison(normal_directory, pcos_directory)
comparison.process_images_in_directory()

```





7. Follicle Overlay and Highlighting (1000 Images)

#6. Follicle Overlay and Highlighting

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os

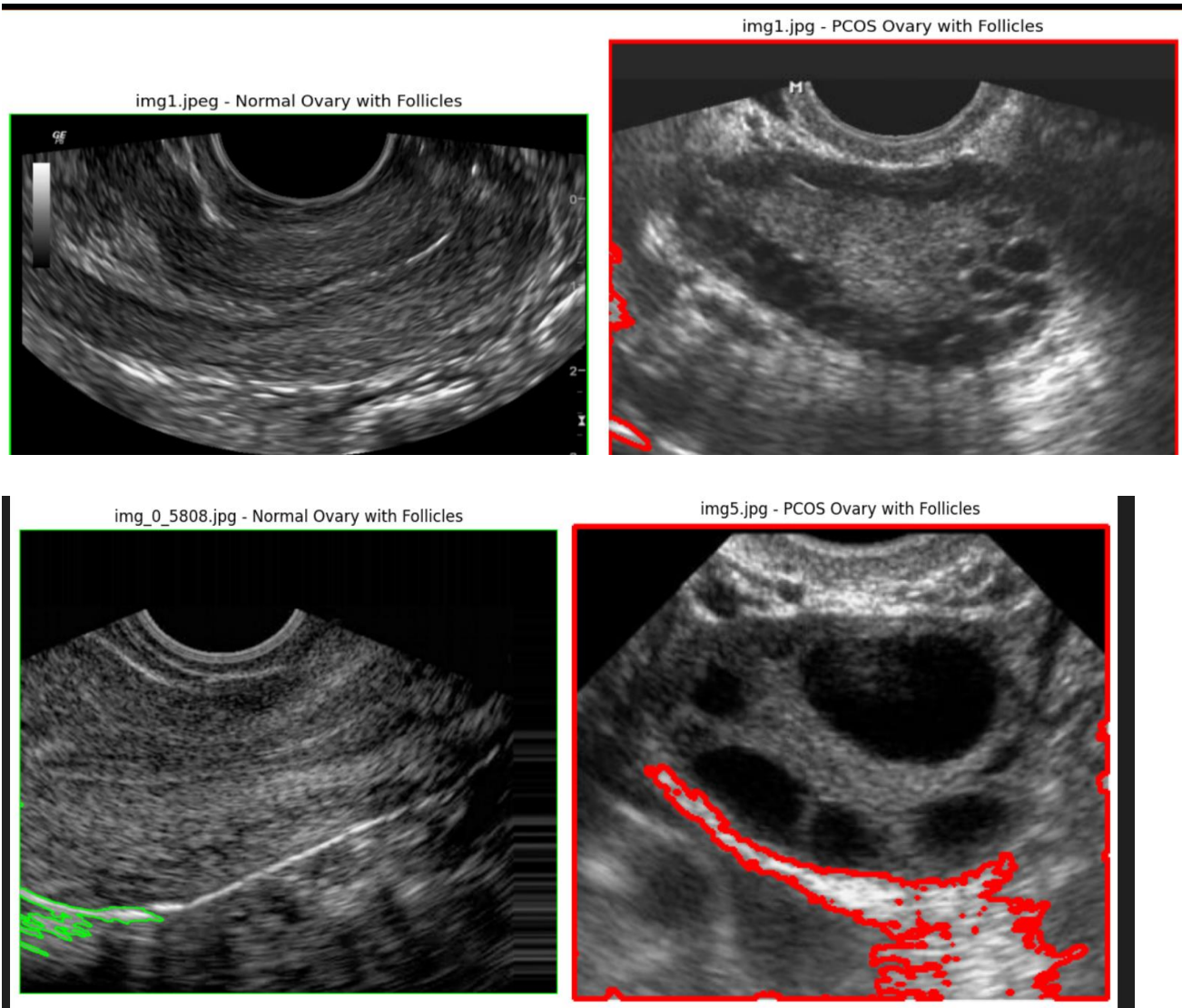
class OvaryAnalysis:
    def __init__(self, normal_directory, pcpos_directory):
        self.normal_directory = normal_directory
        self.pcos_directory = pcpos_directory
        self.normal_images = sorted(os.listdir(normal_directory))
        self.pcos_images = sorted(os.listdir(pcpos_directory))

    def load_images(self, normal_image_path, pcpos_image_path):
        self.normal_image = cv2.imread(normal_image_path, cv2.IMREAD_GRAYSCALE)
        self.pcos_image = cv2.imread(pcpos_image_path, cv2.IMREAD_GRAYSCALE)

    def process_image(self, image):
        _, thresh = cv2.threshold(image, 120, 255, cv2.THRESH_BINARY_INV)
        contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
        return contours, thresh

    def analyze_images(self):
        normal_contours, _ = self.process_image(self.normal_image)
        pcpos_contours, _ = self.process_image(self.pcos_image)

        self.normal_overlay = cv2.cvtColor(self.normal_image, cv2.COLOR_GRAY2RGB)
        self.pcos_overlay = cv2.cvtColor(self.pcos_image, cv2.COLOR_GRAY2RGB)
        cv2.drawContours(self.normal_overlay, normal_contours, -1, (0, 255, 0), 2)
        cv2.drawContours(self.pcos_overlay, pcpos_contours, -1, (255, 0, 0), 2)
```



This code processes and analyses ovary images to highlight follicles and compare normal ovaries with PCOS-affected ones. It loads grayscale images, applies binary thresholding to detect follicular contours, overlays these contours (green for normal, blue for PCOS), and visualizes the results side-by-side using Matplotlib. This helps in studying and diagnosing structural differences in follicular patterns for medical research or diagnostics.

8. Error Handling (1000 Images)

```
import os

def safe_image_processing(image_path):
    try:
        # Placeholder for actual image processing logic
        processed_image = image_path
        return processed_image
    except Exception as e:
        print("Error processing image:", image_path, "-", e)
        return None

def process_dataset(image_folder):
    processed_images = []
    try:
        for image_file in os.listdir(image_folder):
            try:
                image_path = os.path.join(image_folder, image_file)
                if not image_file.lower().endswith(('.png', '.jpg', '.jpeg', '.bmp')):
                    print("Skipping unsupported file:", image_file)
                    continue
                processed_image = safe_image_processing(image_path)
                if processed_image is not None:
                    processed_images.append(processed_image)
            else:
                print("Failed to process image:", image_file)
    except Exception as e:
        print("Error handling file:", image_file, "-", e)
    except Exception as e:
        print("Error accessing folder:", image_folder, "-", e)
    return processed_images

image_folder = '/content/drive/MyDrive/extracted_files_2'
processed_images_dataset = process_dataset(image_folder)

print("Processed", len(processed_images_dataset), "images successfully.")
```



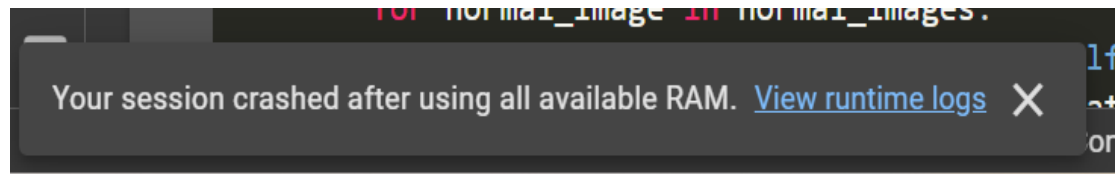
```
Skipping unsupported file: dataset
Processed 0 images successfully.
```

8.Challenges Faced During Implementation **of Ultrasound Images and Data Analysis**

1. Runtime Errors in Colab:

Large datasets caused session crashes due to exceeding Colab's memory and runtime limits.

Processing each cell took 20-25 minutes, making the workflow inefficient.



2. Memory Overload:

Handling huge ultrasound image datasets led to RAM crashes and session resets.

3. Image Conversion Issues:

Converting grayscale ultrasound images to RGB was unsuitable as the images lacked color information, making overlays less effective.

4. Binary Visualization Difficulties:

Binary thresholding caused challenges in accurately visualizing follicles due to loss of finer details or unclear boundaries in black-and-white format.

5. Inaccurate Correlation :with strongly affecting Hormones(FSH,LH,TSH) with PCOS (Y/N) which made the further analysis useless and dataset was changed.

Results Summary

1. **Dataset Overview:** The dataset contains 500 normal and 500 PCOS ovarian images, balanced for analysis.

2. **Image Preprocessing:**

Ovarian regions were successfully isolated, and images were augmented (flipped, rotated, scaled).

Canny edge detection highlighted clear ovarian boundaries.

Contour drawing identified follicle structures.

3. **Pixel Intensity Comparison:**

Normal ovaries showed less intensity variation

PCOS ovaries showed higher intensity variation .

4. **Image Transformations:**

Converted images to binary, grayscale, and RGB, enhancing visualization.

6. **Heatmap Comparison:**

Heatmaps showed normal ovaries with evenly distributed follicles and PCOS ovaries with dense follicle clustering.

7. **Follicle Overlay:**

Normal ovaries had fewer, evenly distributed follicle markers, while PCOS ovaries showed dense clusters.

8. **DataFrame Creation:**

Pixel intensity data was converted into a data-frame for analysis.

Output: Clear visual and statistical differences between normal and PCOS ovaries, including follicle distribution, intensity variation, and clustering patterns to help Doctors more effectively.

9.Discussion

1.Interpretation of Results:

Image Preprocessing & Augmentation: The augmentation techniques (flipping, rotating, scaling) allowed for a more comprehensive analysis of the ovaries by increasing the diversity of the dataset. Canny edge detection and contour drawing effectively highlighted ovarian boundaries, aiding in follicle identification.

Pixel Intensity Comparison: The pixel intensity histograms of normal and PCOS ovaries revealed distinct patterns. Normal ovaries showed a narrower range of pixel intensities with lower variation, indicating more uniform follicle distribution. PCOS ovaries, on the other hand, had a wider range and higher variation in pixel intensities, reflecting irregular follicle sizes and clustering, characteristic of PCOS.

Image Transformation: Converting images into different formats (binary, grayscale, RGB) provided varied views, enhancing the understanding of follicle distribution and ovarian structure.

Heatmap & Follicle Overlay: Heatmaps highlighted differences in follicle distribution, with normal ovaries showing more even distribution and PCOS ovaries displaying clustering. Follicle overlays further emphasized these differences, making it easier to visually distinguish normal and PCOS ovaries.

DataFrame Creation: Converting pixel data to a DataFrame allowed for deeper analysis and comparison of pixel intensities, providing valuable quantitative data to complement the visual results.

2.Significance in Biological Context:

The variations observed in the analysis are biologically significant. In normal ovaries, the follicles are typically smaller and more evenly distributed, whereas in PCOS ovaries, the follicles are often larger and form clusters, leading to cyst formation. The techniques used in this study can provide insights into ovarian health, contributing to the early detection of PCOS and potentially offering a non-invasive diagnostic tool for clinicians.

3.Limitations:

Image Quality: Low-resolution or noisy images could affect the accuracy of the analysis, especially in contour detection and intensity measurement.

Thresholding Sensitivity: The choice of threshold values in the Canny edge detection and binary conversion may influence the accuracy of follicle identification. Variations in ovarian size or image contrast may require fine-tuning of these thresholds.

Dataset Bias: The results are based on a specific dataset of normal and PCOS ovaries, which may not fully represent the wide variability in PCOS presentations across different individuals. A more diverse dataset could improve the generalizability of the findings.

Computational Resources: Processing large datasets can be resource-intensive, and limitations in computational power (ex RAM, processing time) may affect the efficiency and scalability of the analysis.

10.Conclusion

Summary of Work

This study aimed to develop an effective methodology for analysing and differentiating normal and PCOS ovaries using ultrasound image data. The workflow incorporated several critical steps, including image preprocessing, augmentation, pixel intensity analysis, transformation into different formats, heatmap visualization, and follicle overlay techniques. The analysis leveraged contour detection, pixel intensity distributions, and statistical metrics to discern structural differences between normal and PCOS ovaries. Furthermore, the conversion of image data into tabular format enabled deeper quantitative analysis.

Key Takeaways

1. **Distinct Pixel Intensity Distributions:** The analysis revealed that normal ovaries exhibit a narrower range of pixel intensities, indicating a more uniform follicle distribution. In

contrast, PCOS ovaries displayed a broader intensity range with greater variation, reflecting irregular follicle clustering and cyst formation.

2. **Enhanced Visualization:** The application of heatmaps and follicle overlays effectively highlighted the structural differences between normal and PCOS ovaries, providing clear visual cues for ovarian health assessment.
3. **Quantitative Insights:** By converting pixel data into a DataFrame, the study facilitated a more detailed comparison of the pixel intensities, offering statistical insights that complement visual analysis.

Potential Applications

- **PCOS Diagnosis:** This methodology can be adapted into diagnostic tools to assist clinicians in the early detection of PCOS, providing a non-invasive and efficient approach to diagnosis.
- **Ovarian Health Monitoring:** Regular monitoring of ovarian structures using this technique could aid in tracking the progression of PCOS or other ovarian disorders, enabling better treatment planning.
- **Medical Research:** This approach provides a valuable framework for research into ovarian health, enabling further study of follicular dynamics and the pathophysiology of PCOS.

Suggestions for Future Work

1. **Improved Image Quality:** Incorporating higher-resolution ultrasound images and advanced noise reduction techniques could enhance the accuracy and reliability of the analysis.
2. **Advanced Segmentation and Thresholding:** Employing adaptive thresholding methods or advanced segmentation algorithms could improve the detection of follicles and ovarian boundaries, especially in images with varying contrast and quality.

3. **Expanding Dataset Diversity:** The inclusion of a broader and more varied dataset, encompassing different stages of PCOS and a wider range of image qualities, would help to generalize the findings and enhance the robustness of the model.
4. **Real-time Processing:** Optimizing the workflow for faster processing would enable its implementation in real-time clinical settings, providing timely insights for healthcare professionals.
5. **Integration with Machine Learning:** Leveraging machine learning techniques could automate the detection and classification of follicles, further improving accuracy and efficiency in diagnosing ovarian disorders like PCOS.

In conclusion, this study presents a comprehensive approach to analyzing ovarian structures, with significant potential in the clinical and research domains. Future work focused on refining image processing techniques, expanding datasets, and integrating advanced algorithms will enhance the utility and applicability of this workflow for PCOS detection and monitoring.

11. References

1. Rocha, A. L., Oliveira, F. R., Azevedo, R. C., Silva, V. A., Peres, T. M., Candido, A. L., Gomes, K. B., & Reis, F. M. (2019). *Recent advances in the understanding and management of polycystic ovary syndrome [Version 1; peer review: 3 approved]. F1000Research, 8, 565.*
<https://doi.org/10.12688/f1000research.15318.1>
2. C., M., Nishad, K. M., Umunnoora, K. P., Jushna, K., Saheera, K. V., & Dilsha, K. P. (2018). *Polycystic ovary syndrome (PCOS)—an overview. Vol. 10, Issue 6, Review Article. Jamiasalafiya Pharmacy College, Pulikkal, Kerala, India. Received: 10 Jul 2018, Revised and Accepted: 07 Sep 2018.*
3. Rababa'h, A. M., Matania, B. R., & Yehya, A. (Year). *An update of polycystic ovary syndrome: Causes and therapeutic options. Journal Name, Volume(Issue), pages. Department of Clinical Pharmacy, College of Pharmacy, Jordan University of Science and Technology, Irbid, Jordan; Department of Clinical Pharmacy and Pharmacy Practice Faculty of Pharmacy, Yarmouk University, Irbid, Jordan.*

4. Gonzalez, R. C., & Woods, R. E. (2008). *Digital image processing (3rd ed.)*. Pearson Prentice Hall.
5. Jain, A. K. (1989). *Fundamentals of digital image processing*. Prentice-Hall.
6. Castleman, K. R. (1996). *Digital image processing*. Prentice Hall.

Here are four research papers formatted in APA style:

7. A Novel Approach for Polycystic Ovary Syndrome Prediction Using Machine Learning
Nature Scientific Reports. (2022). Retrieved from <https://www.nature.com/articles/s41598-022-21724-0>

8. Development of a Machine Learning-Based Model for Accurate PCOS Detection
PMC (PubMed Central). (2022). Retrieved from
<https://pmc.ncbi.nlm.nih.gov/articles/PMC11338641>

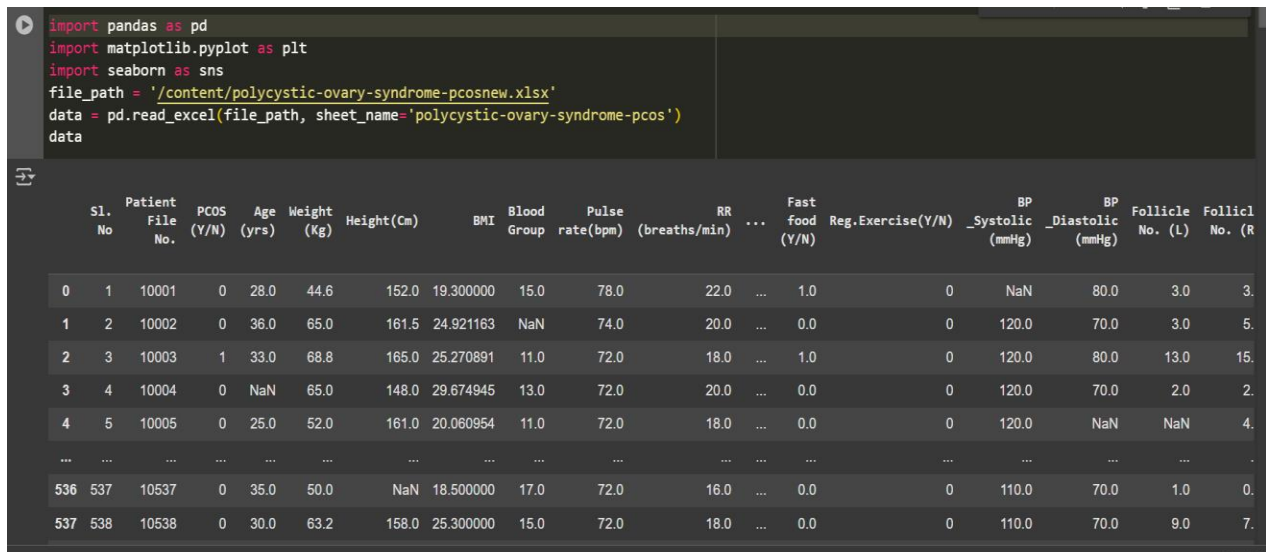
9. Identification of Polycystic Ovary Syndrome in Ultrasound Images of Ovaries Using Distinct
Threshold-Based Image Segmentation
IEEE Xplore Digital Library. (2023). Retrieved from
<https://ieeexplore.ieee.org/document/10141800>

10. An Innovative Approach for PCO Morphology Segmentation Using a Hybrid Deep Learning
Model
Springer Link. (2024). Retrieved from <https://link.springer.com/article/10.1007/s10791-024-09458-1>

12.Appendix

The dataset we previously used, *PCOS Analysis* (with 541 rows and 43 columns), was downloaded from Kaggle. However, its small size limits its effectiveness for in-depth analysis, as the relatively low number of rows can lead to less accurate visualizations and statistical insights.

Link: <https://www.kaggle.com/code/mahikkaaa/pcos-prediction-and-analysis?scriptVersionId=132072146&cellId=3>



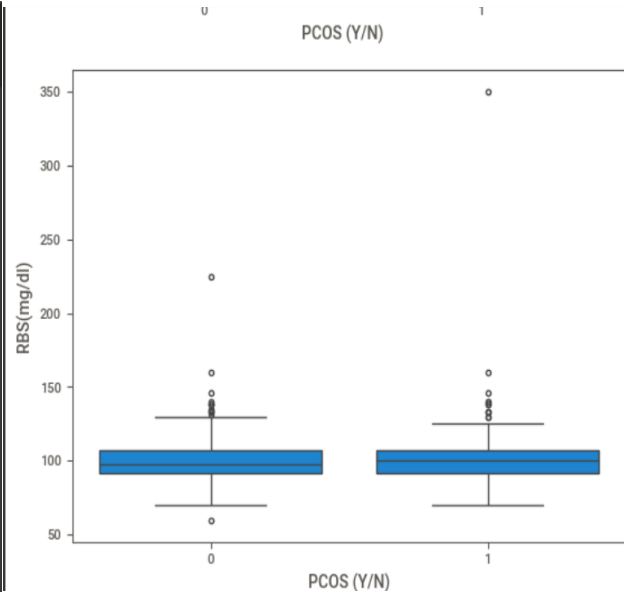
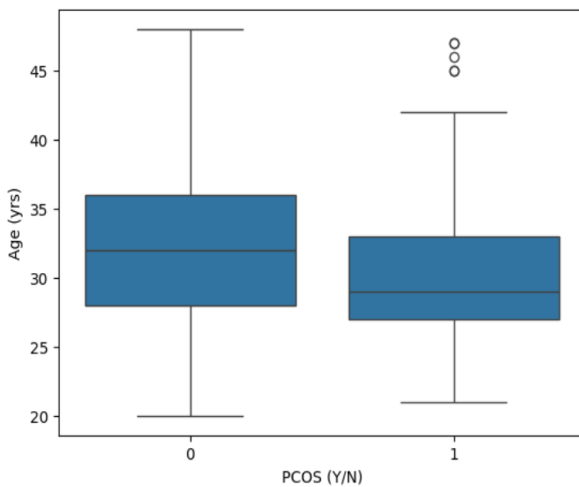
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
file_path = '/content/polycystic-ovary-syndrome-pcosnew.xlsx'
data = pd.read_excel(file_path, sheet_name='polycystic-ovary-syndrome-pcos')
data
```

	Sl. No	Patient File No.	PCOS (Y/N)	Age (yrs)	Weight (Kg)	Height(Cm)	BMI	Blood Group	Pulse rate(bpm)	RR (breaths/min)	...	Fast food (Y/N)	Reg.Exercise(Y/N)	BP _Systolic (mmHg)	BP _Diastolic (mmHg)	Follicle No. (L)	Follicl No. (R)
0	1	10001	0	28.0	44.6	152.0	19.300000	15.0	78.0	22.0	...	1.0	0	NaN	80.0	3.0	3.
1	2	10002	0	38.0	65.0	161.5	24.921163	NaN	74.0	20.0	...	0.0	0	120.0	70.0	3.0	5.
2	3	10003	1	33.0	68.8	165.0	25.270891	11.0	72.0	18.0	...	1.0	0	120.0	80.0	13.0	15.
3	4	10004	0	NaN	65.0	148.0	29.674945	13.0	72.0	20.0	...	0.0	0	120.0	70.0	2.0	2.
4	5	10005	0	25.0	52.0	161.0	20.060954	11.0	72.0	18.0	...	0.0	0	120.0	NaN	NaN	4.
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
536	537	10537	0	35.0	50.0	NaN	18.500000	17.0	72.0	16.0	...	0.0	0	110.0	70.0	1.0	0.
537	538	10538	0	30.0	63.2	158.0	25.300000	15.0	72.0	18.0	...	0.0	0	110.0	70.0	9.0	7.

As we are getting negative values for strongly affecting Parameters for PCOS like hormones FSH, LH, TSH they are negatively correlated to PCOS which is absurd so we have to change our dataset.

PRL(ng/mL)	0.005143
TSH (mIU/L)	-0.010140
FSH/LH	-0.018336
Pregnant(Y/N)	-0.027565
FSH(mIU/mL)	-0.030319
PRG(ng/mL)	-0.043834
No. of absorptions	-0.057158
Marriage Status (Yrs)	-0.113056
Age (yrs)	-0.168513
Cycle length(days)	-0.178480

```
for y in numerical_variables:
    sns.boxplot(y=y, x="PCOS (Y/N)", data=df)
    plt.show()
```



```
df=pd.DataFrame(data)
numerical_df = df.select_dtypes(include=['number'])
corr_matrix=numerical_df.corr()
corr_matrix
```

	Sl. No	Patient File No.	PCOS (Y/N)	Age (yrs)	Weight (Kg)	Height(Cm)	BMI	Blood Group	Pulse rate(bpm)	RR (breaths/min)	...	Fast food (Y/N)	Reg.Exercise(Y/N)	_Sys (
Sl. No	1.000000	1.000000	0.060998	0.076269	-0.084183	-0.017135	-0.078648	0.001938	-0.041600	-0.012785	...	0.154135	0.000110	0.0
Patient File No.	1.000000	1.000000	0.060998	0.076269	-0.084183	-0.017135	-0.078648	0.001938	-0.041600	-0.012785	...	0.154135	0.000110	0.0
PCOS (Y/N)	0.060998	0.060998	1.000000	-0.160608	0.208978	0.072244	0.204741	0.033556	0.093623	0.039874	...	0.377933	0.065337	0.0
Age (yrs)	0.076269	0.076269	-0.160608	1.000000	-0.015186	-0.113042	0.023431	-0.008712	0.044740	0.089864	...	-0.031660	-0.023837	0.0
Weight (Kg)	-0.084183	-0.084183	0.208978	-0.015186	1.000000	0.433868	0.900753	0.077571	0.022272	0.049434	...	0.165180	-0.068758	0.0
Height(Cm)	-0.017135	-0.017135	0.072244	-0.113042	0.433868	1.000000	-0.004273	0.034365	-0.071918	-0.019402	...	0.036355	0.015477	-0.0
BMI	-0.078648	-0.078648	0.204741	0.023431	0.900753	-0.004273	1.000000	0.057013	0.048536	0.064025	...	0.166447	-0.080659	0.0
Blood Group	0.001938	0.001938	0.033556	-0.008712	0.077571	0.034365	0.057013	1.000000	0.049796	-0.017503	...	-0.027407	0.061432	0.0
Pulse rate(bpm)	-0.041600	-0.041600	0.093623	0.044740	0.022272	-0.071918	0.048536	0.049796	1.000000	0.302720	...	0.057274	-0.039042	-0.0
RR (breaths/min)	-0.012785	-0.012785	0.039874	0.089864	0.049434	-0.019402	0.064025	-0.017503	0.302720	1.000000	...	0.141596	-0.046729	0.0
Hb(g/dl)	-0.088913	-0.088913	0.090024	-0.029748	0.012824	0.027350	0.008973	-0.015641	-0.054218	-0.036923	...	0.079052	0.011910	0.0

```
print(data.info())
print(data.describe())
print(data.columns)
```

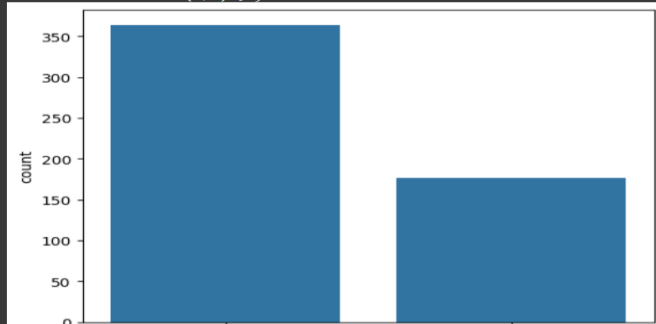
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 541 entries, 0 to 540
Data columns (total 43 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Sl. No                                541 non-null    int64
1   Patient File No.                     541 non-null    int64
2   PCOS (Y/N)                           541 non-null    int64
3   Age (yrs)                            532 non-null    float64
4   Weight (Kg)                          533 non-null    float64
5   Height(Cm)                           533 non-null    float64
6   BMI                                   535 non-null    float64
7   Blood Group                          534 non-null    float64
8   Pulse rate(bpm)                      537 non-null    float64
9   RR (breaths/min)                    536 non-null    float64
10  Hb(g/dl)                             534 non-null    float64
11  Cycle(R/I)                           541 non-null    int64
12  Cycle length(days)                   541 non-null    int64
13  Marraige Status (Yrs)                 540 non-null    float64
14  Pregnant(Y/N)                        541 non-null    int64
15  No. of aborptions                     541 non-null    int64
16  FSH(mIU/mL)                          533 non-null    float64
17  LH(mIU/mL)                           536 non-null    float64
18  FSH/LH                               528 non-null    float64
19  Hip(inch)                            541 non-null    int64
```

weight gain, hair growth, pimples, hair loss, fast food and skin darkening means higher chances of PCOS.

So the above can also be termed as symptoms of PCOS

```
sns.countplot(x = "PCOS (Y/N)", data = df)
```

<Axes: xlabel='PCOS (Y/N)', ylabel='count'>



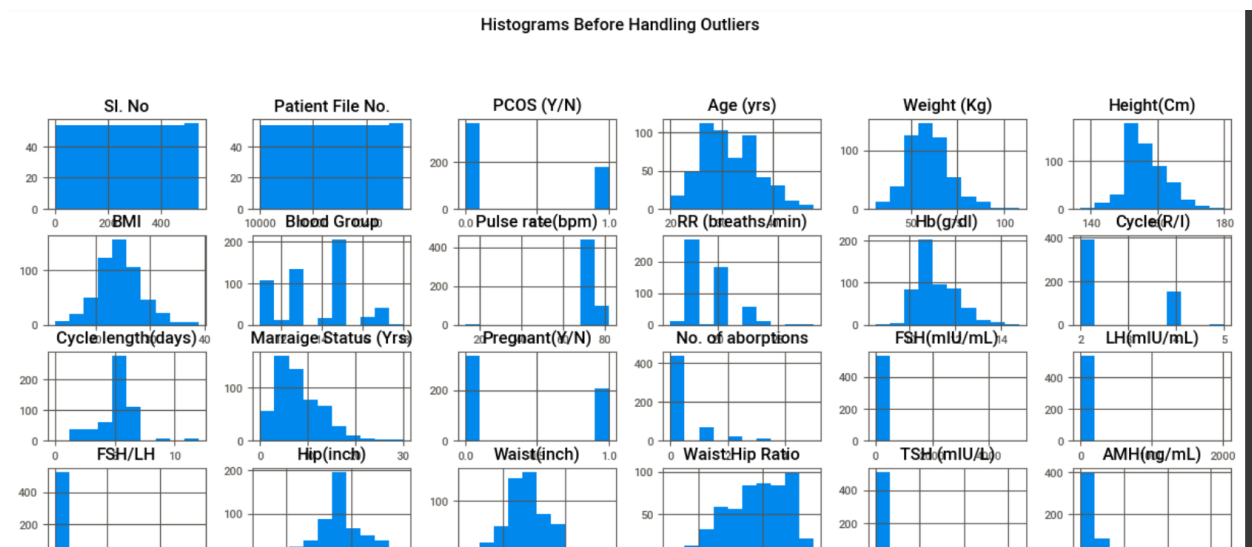
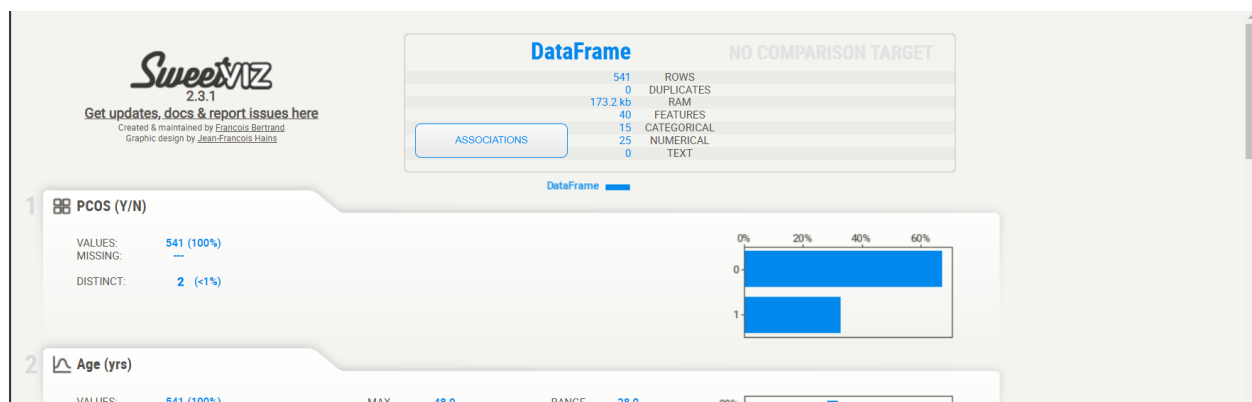
```
df["PCOS (Y/N)"].value_counts()

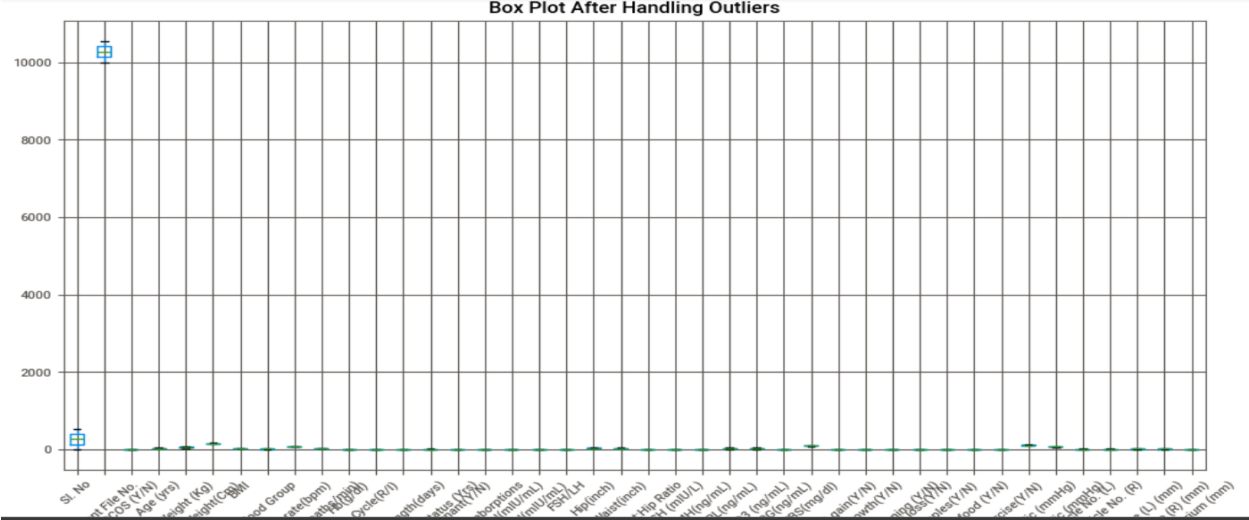
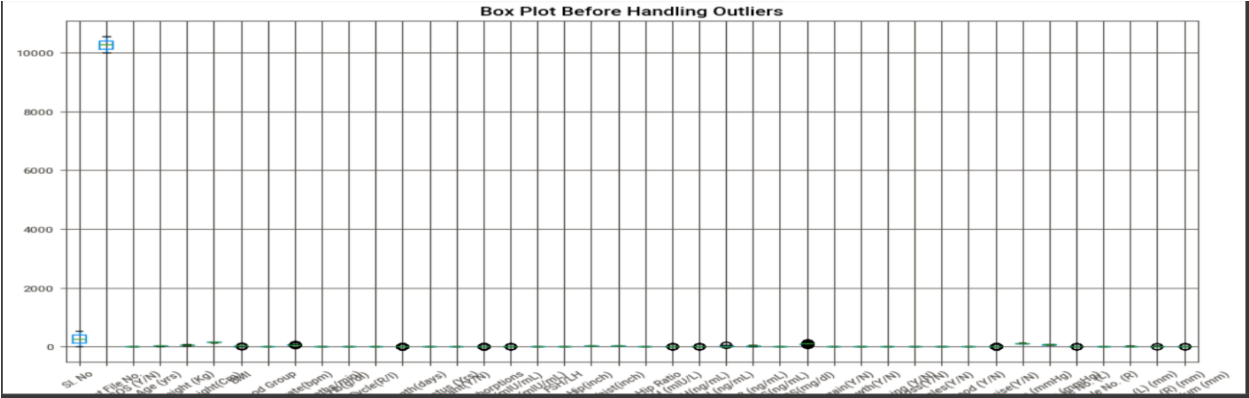
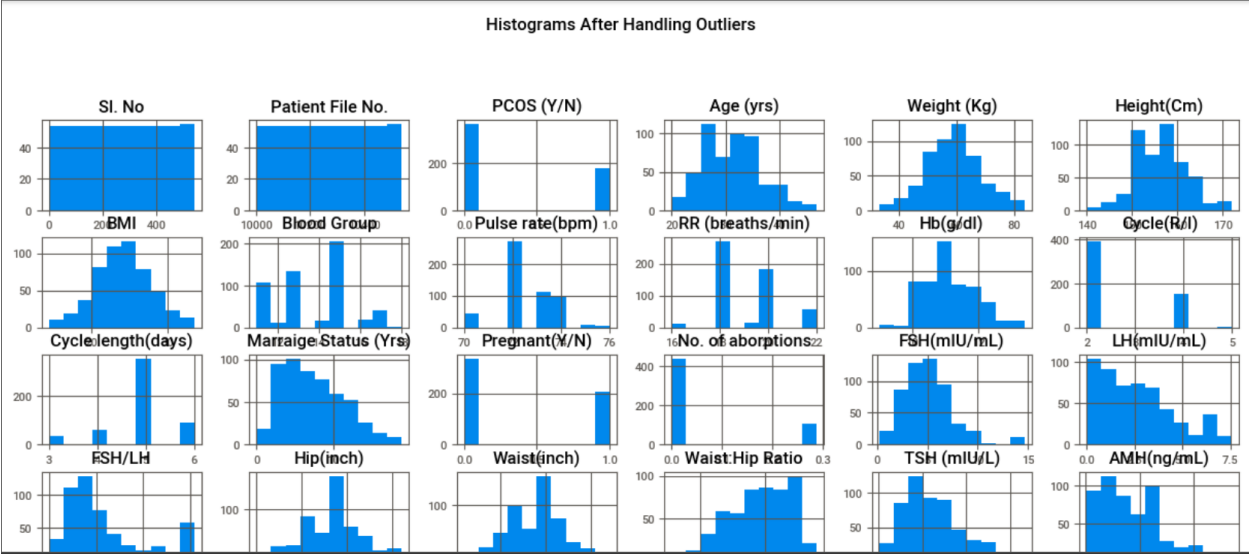
count
PCOS (Y/N)
0      364
1      177

dtype: int64

177/(364 + 177)*100

32.71719038817005
```





std	6.064875	4.058609	1.836618	4.443790
min	137.000000	12.417882	11.000000	13.000000
25%	152.000000	21.641274	13.000000	72.000000
50%	156.000000	24.218750	14.500000	72.000000
75%	160.000000	26.531250	15.000000	74.000000
max	180.000000	38.900000	18.000000	82.000000

	RR (breaths/min)	...	Fast food (Y/N)	Reg.Exercise(Y/N)	\
count	536.000000	...	540.000000	541.000000	
mean	19.244403	...	0.514815	0.247689	
std	1.693850	...	0.500244	0.432070	
min	16.000000	...	0.000000	0.000000	
25%	18.000000	...	0.000000	0.000000	
50%	18.000000	...	1.000000	0.000000	
75%	20.000000	...	1.000000	0.000000	
max	28.000000	...	1.000000	1.000000	

	BP _Systolic (mmHg)	BP _Diastolic (mmHg)	Follicle No. (L)	\
count	531.000000	532.000000	536.000000	
mean	114.655367	76.875940	6.136194	
std	7.371021	5.606663	4.228311	
min	12.000000	8.000000	0.000000	
25%	110.000000	70.000000	3.000000	
50%	110.000000	80.000000	5.000000	
75%	120.000000	80.000000	9.000000	

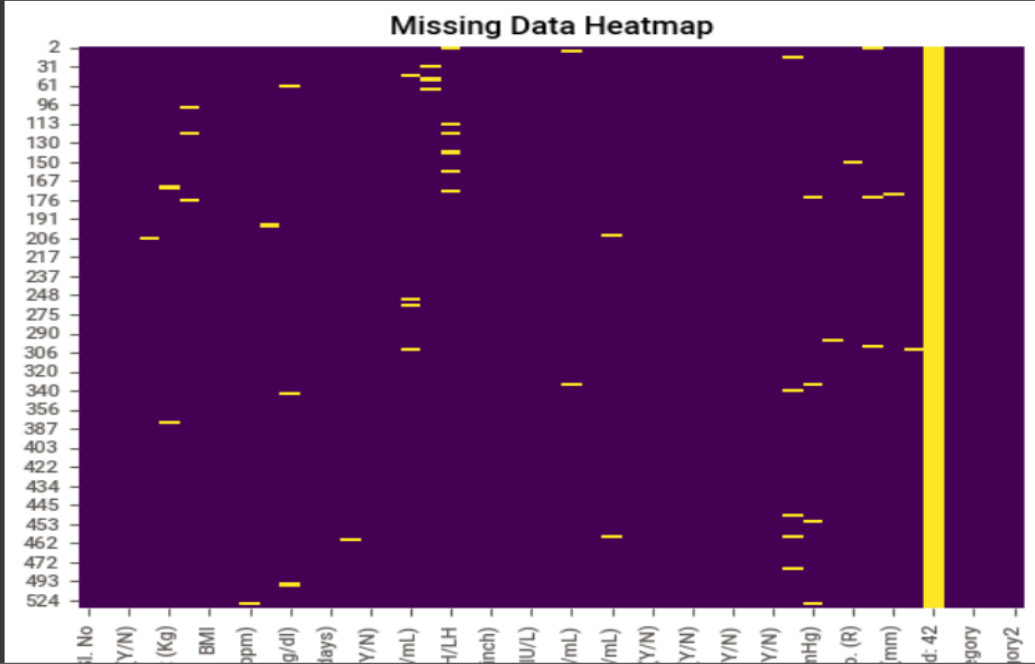
[8 rows x 43 columns]

```
Index(['Sl. No', 'Patient File No.', 'PCOS (Y/N)', ' Age (yrs)', 'Weight (Kg)',
      'Height(Cm) ', 'BMI', 'Blood Group', 'Pulse rate(bpm) ',
      'RR (breaths/min)', 'Hb(g/dl)', 'Cycle(R/I)', 'Cycle length(days)',
      'Marraige Status (Yrs)', 'Pregnant(Y/N)', 'No. of aborptions',
      'FSH(mIU/mL)', 'LH(mIU/mL)', 'FSH/LH', 'Hip(inch)', 'Waist(inch)',
      'Waist:Hip Ratio', 'TSH (mIU/L)', 'AMH(ng/mL)', 'PRL(ng/mL)',
      'Vit D3 (ng/mL)', 'PRG(ng/mL)', 'RBS(mg/dl)', 'Weight gain(Y/N)',
      'hair growth(Y/N)', 'Skin darkening (Y/N)', 'Hair loss(Y/N)',
      'Pimples(Y/N)', 'Fast food (Y/N)', 'Reg.Exercise(Y/N)',
      'BP _Systolic (mmHg)', 'BP _Diastolic (mmHg)', 'Follicle No. (L)',
      'Follicle No. (R)', 'Avg. F size (L) (mm)', 'Avg. F size (R) (mm)',
      'Endometrium (mm)', 'Unnamed: 42'],
      dtype='object')
```

```
print(df.isnull().mean()* 100)
```

Sl. No	0.000000
Patient File No.	0.000000
PCOS (Y/N)	0.000000
Age (yrs)	1.663586
Weight (Kg)	1.478743
Height(Cm)	1.478743
BMI	1.109057
Blood Group	1.293900
Pulse rate(bpm)	0.739372
RR (breaths/min)	0.924214
Hb(g/dl)	1.293900
Cycle(R/I)	0.000000
Cycle length(days)	0.000000
Marraige Status (Yrs)	0.184843
Pregnant(Y/N)	0.000000
No. of absorptions	0.000000
FSH(mIU/mL)	1.478743
LH(mIU/mL)	0.924214
FSH/LH	2.402957
Hip(inch)	0.000000
Waist(inch)	0.000000
Waist:Hip Ratio	0.000000
TSH (mIU/L)	0.000000
AMH(ng/mL)	0.184843
PRL(ng/mL)	0.924214
Vit D3 (ng/mL)	0.000000
PRG(ng/mL)	1.109057
RBS(mg/dl)	0.739372


```
sns.heatmap(pcos_females.isnull(), cbar=False, cmap='viridis')
plt.title('Missing Data Heatmap')
plt.show()
```



```
#Plotting correlation heatmap
plt.figure(figsize=(20, 20))
# plotting correlation heatmap
dataplot = sns.heatmap(df.corr(), cmap="spring", annot=True)

# displaying heatmap
plt.show()
```



7. Convert Image to DataFrame (10 images)

```
#7.Convert Image to DataFrame
import cv2
import pandas as pd
import numpy as np
import os

class ImageAnalysis:
    def __init__(self, normal_directory, pcpos_directory):
        self.normal_directory = normal_directory
        self.pcos_directory = pcpos_directory
        self.normal_dataframes = []
        self.pcos_dataframes = []

    def convert_image_to_dataframe(self, image_path):
        image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
        image_flat = image.flatten()
        return pd.DataFrame(image_flat, columns=['Pixel Intensity'])

    def process_images(self):
        normal_images = sorted(os.listdir(self.normal_directory))
        pcpos_images = sorted(os.listdir(self.pcos_directory))

        for normal_image in normal_images:
            normal_image_path = os.path.join(self.normal_directory, normal_image)
            normal_df = self.convert_image_to_dataframe(normal_image_path)
            normal_df['Class'] = 'Normal'
            self.normal_dataframes.append(normal_df)

        for pcpos_image in pcpos_images:
            pcpos_image_path = os.path.join(self.pcos_directory, pcpos_image)
            pcpos_df = self.convert_image_to_dataframe(pcpos_image_path)
            pcpos_df['Class'] = 'PCOS'
            self.pcos_dataframes.append(pcpos_df)
```

```
def analyze_dataset(self):
    all_data = pd.concat(self.normal_dataframes + self.pcos_dataframes, ignore_index=True)
    return all_data.groupby('Class')['Pixel Intensity'].describe()

normal_directory = '/content/drive/MyDrive/extracted_files_2/dataset/normal'
pcpos_directory = '/content/drive/MyDrive/extracted_files_2/dataset/pcos'
analysis = ImageAnalysis(normal_directory, pcpos_directory)
analysis.process_images()
statistics = analysis.analyze_dataset()
print(statistics)
```

The ImageAnalysis class processes normal and PCOS ovary images by converting them into grayscale and flattening the pixel intensities into DataFrames. Each image is labeled with its respective class ('Normal' or 'PCOS'), and the pixel intensity data is stored in separate lists. After processing, the data is combined into a single DataFrame, and descriptive statistics for pixel intensity are computed for both normal and PCOS images. The result gives an overview of the pixel intensity distributions for each class.

Pixel Intensity Statistics by Class (Normal vs. PCOS):

	count	mean	std	min	25%	50%	75%	max
Class								
Normal	660000.0	74.455597	45.624430	0.0	36.0	68.0	103.0	255.0
PCOS	600000.0	70.891578	43.625304	0.0	35.0	67.0	101.0	255.0

8. Visualization and Analysis (for 1 image)

```
import matplotlib.pyplot as plt
import numpy as np
import cv2

original_image = cv2.imread("/content/normal.jpg")
augmented_image = cv2.imread("/content/augmented.jpg")

if original_image.shape != augmented_image.shape:
    augmented_image = cv2.resize(augmented_image, (original_image.shape[1], original_image.shape[0]))

original_image = cv2.cvtColor(original_image, cv2.COLOR_BGR2RGB)
augmented_image = cv2.cvtColor(augmented_image, cv2.COLOR_BGR2RGB)

def plot_comparison(original, augmented):
    plt.figure(figsize=(10, 5))

    plt.subplot(1, 2, 1)
    plt.imshow(original)
    plt.title("Original Image")
    plt.axis("off")

    plt.subplot(1, 2, 2)
    plt.imshow(augmented)
    plt.title("Augmented Image")
    plt.axis("off")

    plt.tight_layout()
    plt.show()
```

```

def visualize_differences(original, augmented):
    difference = cv2.absdiff(original, augmented)
    grayscale_diff = cv2.cvtColor(difference, cv2.COLOR_RGB2GRAY)

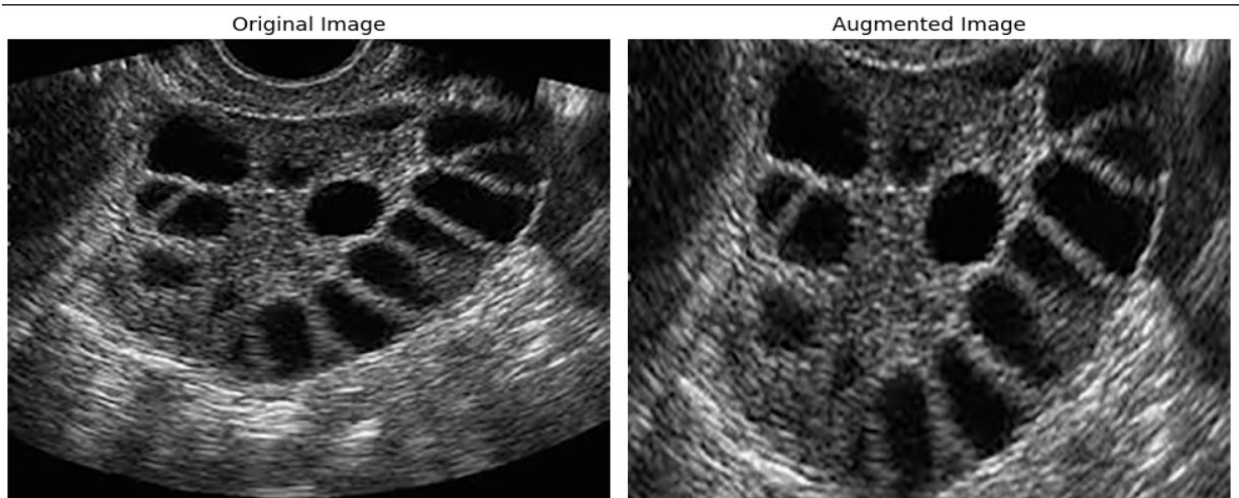
    plt.figure(figsize=(8, 6))
    plt.imshow(grayscale_diff, cmap='hot')
    plt.title("Differences in Ovarian Structure")
    plt.colorbar(label="Difference Intensity")
    plt.axis("off")
    plt.show()

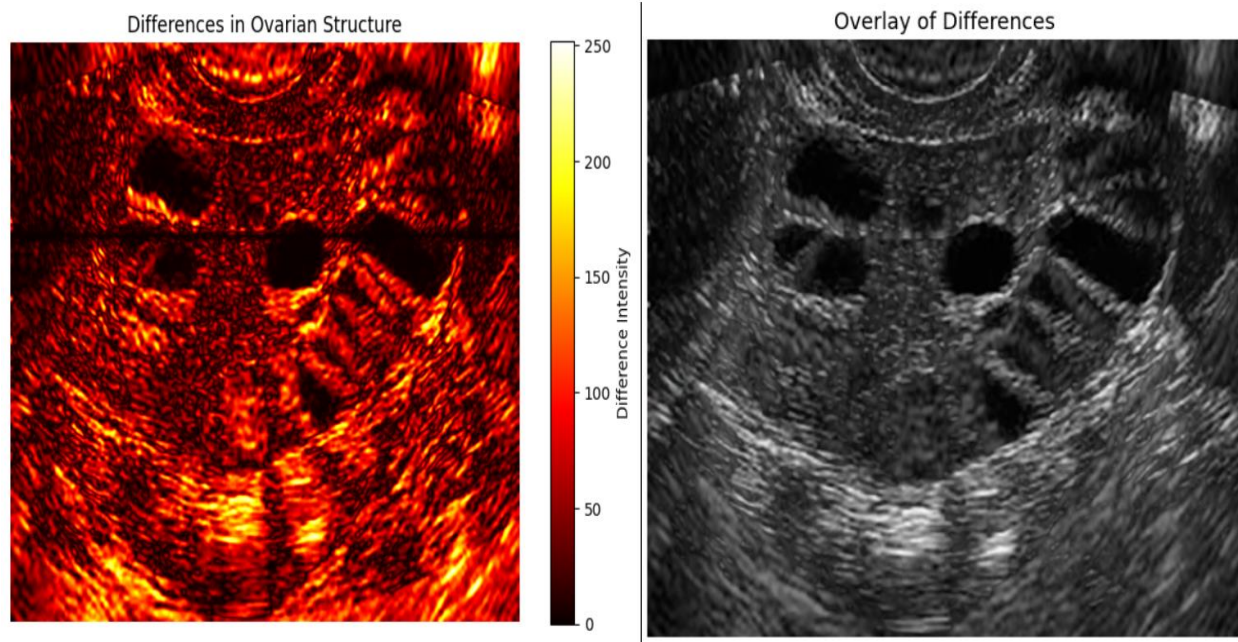
def overlay_differences(original, augmented, alpha=0.5):
    difference = cv2.absdiff(original, augmented)
    overlay = cv2.addWeighted(original, 1 - alpha, difference, alpha, 0)

    plt.figure(figsize=(8, 6))
    plt.imshow(overlay)
    plt.title("Overlay of Differences")
    plt.axis("off")
    plt.show()

plot_comparison(original_image, augmented_image)
visualize_differences(original_image, augmented_image)
overlay_differences(original_image, augmented_image)

```





The code compares and visualizes two images: an original and an augmented one. It starts by loading the images using OpenCV (`cv2.imread()`) and ensures both images are the same size by resizing the augmented image to match the original. Then, the images are converted from OpenCV's BGR format to RGB for proper display using `cv2.cvtColor()`. Three main functions are used: `plot_comparison()` displays both images side by side, `visualize_differences()` computes the absolute differences between the images and visualizes these differences in a heatmap, and `overlay_differences()` overlays the difference on top of the original image with transparency. The overall output helps in visually analyzing the changes or differences between the two images.